

```
#!/usr/bin/env python3
```

```
"""
```

```
mame_listener.py - captures one MAME test run's serial output over TCP.
```

```
Usage:
```

```
    mame_listener.py PORT LOGFILE [--idle-timeout SECS] [--hard-timeout SECS]
```

```
Design notes:
```

- MAME's "-bitb socket.HOST:PORT" first tries to CONNECT to that address as a client; only if nothing is listening does it fall back to listening itself. So this script binds and listens *before* MAME is launched, and MAME connects straight into it - no PTY device path to discover, no race to lose.
- There is no fixed "N bytes expected" signal to wait for: each glossary section's test group has a different number of tests and a different final test name. Instead this uses an IDLE TIMEOUT: once real test output stops arriving (TSTRUNNER has returned and control has fallen through to the normal Forth prompt, which produces no further output on its own), a few seconds of silence reliably means the run is done.
- A separate HARD TIMEOUT is also enforced. This session's own history includes a real BASE=0 infinite-loop bug that hung a test indefinitely (found via MAME, not caught by any static check) - an idle-timeout alone isn't enough if a test hangs while a partial line is still queued or garbled. The hard timeout guarantees this script (and the run) always terminates even if a future test hangs the same way.
- All bytes received are appended to LOGFILE exactly as received, plus a one-line summary is printed to stdout when the connection ends, so the driving loop can tell at a glance whether it looks complete.

```
"""
```

```
import argparse
```

```
import socket
```

```
import sys
```

```
import time
```

```
def main():
```

```
    parser = argparse.ArgumentParser(description=__doc__)
```

```
    parser.add_argument("port", type=int)
```

```
    parser.add_argument("logfile")
```

```

parser.add_argument("--idle-timeout", type=float, default=5.0,
                    help="seconds of silence after which the run is consider
parser.add_argument("--hard-timeout", type=float, default=60.0,
                    help="absolute maximum seconds to wait, regardless of ac
parser.add_argument("--host", default="127.0.0.1")
args = parser.parse_args()

srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
srv.bind((args.host, args.port))
srv.listen(1)
srv.settimeout(args.hard_timeout)

print(f"[listener] listening on {args.host}:{args.port}, waiting for MAME to
      file=sys.stderr, flush=True)

try:
    conn, addr = srv.accept()
except socket.timeout:
    print(f"[listener] FAIL: no connection within {args.hard_timeout}s "
          f"(MAME never connected - check the -bitb address/port match)",
          file=sys.stderr, flush=True)
    sys.exit(2)

print(f"[listener] connected from {addr}", file=sys.stderr, flush=True)
conn.settimeout(1.0) # short recv timeout so we can check idle/hard limits f

start_time = time.monotonic()
last_data_time = start_time
total_bytes = 0

with open(args.logfile, "wb") as f:
    while True:
        now = time.monotonic()
        if now - start_time > args.hard_timeout:
            print(f"[listener] hard timeout ({args.hard_timeout}s) reached -
                  f"stopping regardless of activity (possible hang in the ROM
                  file=sys.stderr, flush=True)
            break
        if now - last_data_time > args.idle_timeout:
            print(f"[listener] idle timeout ({args.idle_timeout}s of silence)
                  f"treating run as complete", file=sys.stderr, flush=True)
            break
        try:
            chunk = conn.recv(4096)
        except socket.timeout:
            continue

```

```

except ConnectionResetError:
    print("[listener] connection reset by MAME (likely process exit)"
          file=sys.stderr, flush=True)
    break
if not chunk:
    print("[listener] connection closed by MAME", file=sys.stderr, fl
          break
f.write(chunk)
f.flush()
total_bytes += len(chunk)
last_data_time = now

conn.close()
srv.close()

elapsed = time.monotonic() - start_time
print(f"[listener] done: {total_bytes} bytes captured in {elapsed:.1f}s -> {a
      file=sys.stderr, flush=True)

# Exit code signals the driving loop whether this looks like a clean
# completion (some bytes captured) or a suspicious empty run.
sys.exit(0 if total_bytes > 0 else 1)

if __name__ == "__main__":
    main()

```